La Biblia del Proyecto

Automatización del Reporte ROI Telcel — de cero a terminado

Guía de estudio para Mauricio

Octubre 2026

Contents

Cómo usar este documento **2**

PARTE 0 — Fundamentos de Python que vas a necesitar **3**
- 0.1 — Variables . . . 3
- 0.2 — Tipos de datos básicos . . . 3
- 0.3 — Listas . . . 3
- 0.4 — Diccionarios . . . 3
- 0.5 — Sets (conjuntos) . . . 3
- 0.6 — Funciones . . . 4
- 0.7 — Condicionales . . . 4
- 0.8 — Loops (`for`) . . . 4
- 0.9 — Imports . . . 4
- 0.10 — Entorno virtual (venv) y pip . . . 4

PARTE 1 — Las librerías del proyecto y para qué sirve cada una **5**

PARTE 2 — Arquitectura del proyecto **5**

PARTE 3 — FASE 0: Contrato de datos **6**
- 3.1 — De dónde salen los nombres de columna . . . 6
- 3.2 — Escribe el set de columnas requeridas . . . 6
- 3.3 — Escribe la función de validación . . . 6
- 3.4 — Diccionario de nombres canónicos (solo para históricos) . . . 6

PARTE 4 — FASE 1: Descarga y preparación de la base **7**
- 4.1 — Autenticación (`auth.py`) . . . 7
- 4.2 — El token se vence — necesitas revisarlo . . . 7
- 4.3 — Pedir las solicitudes de un día . . . 7
- 4.4 — Aplanar cada solicitud a lo que sí necesitas . . . 7
- 4.5 — Convertir fechas (epoch → texto legible) . . . 8
- 4.6 — De lista de diccionarios a tabla (DataFrame) . . . 8
- 4.7 — Crear el periodo (mes canónico) . . . 8
- 4.8 — Detectar y resolver duplicados . . . 8

PARTE 5 — Capa interactiva: elegir el mes y checar la retro **9**
- 5.1 — Preguntar el mes . . . 9
- 5.2 — Calcular el rango de fechas del mes automáticamente . . . 9
- 5.3 — Checar si ya tienes la retro de ese mes . . . 9
- 5.4 — Calcular el % de faltantes contra el umbral del 5% . . . 9

PARTE 6 — FASE 2: Comparativo, envío a Telcel e integración de retro **10**
- 6.1 — Pedir el Comparativo, uno por solicitud . . . 10

6.2 — Procesar en lotes, con pausa . . . 10
6.3 — Cruzar Solicitudes con Comparativo . . . 11
6.4 — Preparar el archivo que se manda a Telcel . . . 11
6.5 — Integrar la retro cuando llega . . . 11

PARTE 7 — FASE 3: Tabla 1 (Conversión) y Tabla 2 (Costo) 11
7.1 — Contar únicos, no filas . . . 11
7.2 — Filtrar antes de agrupar, para “evaluados” . . . 12
7.3 — Calcular la conversión sin dividir entre cero . . . 12
7.4 — Costos desde la hoja de facturación . . . 12

PARTE 8 — FASE 4: Tabla 3 (Resultados) y Tabla 4 (Aprobados) 12
8.1 — Normalizar texto antes de comparar . . . 12
8.2 — Mapear a categorías fijas . . . 12
8.3 — Convertir filas largas en columnas (pivotear) . . . 13
8.4 — Filtrar aprobados . . . 13

PARTE 9 — FASE 5: Tabla 5 (Tasas) y Tabla 6 (Beneficio) 13
9.1 — Alinear dos tablas con índices distintos . . . 13
9.2 — Tasa combinada: suma primero, divide después . . . 14
9.3 — Monto promedio y detección de datos faltantes . . . 14

PARTE 10 — FASE 6: Utilidad y ROI 14
10.1 — Encadenar cálculos con `.assign()` . . . 14
10.2 — Verificación automática . . . 14
10.3 — ROI, protegido contra costo cero . . . 15

PARTE 11 — FASE 7: Generar el Excel final 15
11.1 — Trabajar siempre sobre una copia . . . 15
11.2 — Abrir preservando fórmulas . . . 15
11.3 — Escribir solo las celdas que cambian . . . 15
11.4 — Formato de presentación . . . 15
11.5 — Guardar y verificar . . . 15

PARTE 12 — Glosario rápido de funciones (para consulta) 16

Cierre 17

Cómo usar este documento

Este documento asume que **no sabes programar**. No es una referencia para alguien que ya sabe Python y quiere recordar sintaxis — es una ruta completa: concepto, ejemplo, por qué, cómo lo configuras con tus datos reales, y un checklist antes de avanzar.

No vas a encontrar aquí el programa ya hecho. Vas a encontrar, para cada pieza del programa, la explicación de qué herramienta usar y cómo armarla tú mismo. Si en algún punto copias una línea de ejemplo de aquí y no te funciona, es correcto — significa que tienes que ajustarla a tu caso exacto, no que el documento esté mal.

La meta final: un programa en Python que automatice por completo el **Reporte ROI Telcel**, tal como está documentado en tu proceso actual (el que haces hoy a mano en Excel, 3-9 horas al mes).

PARTE 0 — Fundamentos de Python que vas a necesitar

No necesitas un curso completo de Python. Necesitas estos 9 conceptos, y nada más, para entender todo lo que viene después.

0.1 — Variables

Una variable es una etiqueta que le pones a un valor para poder usarlo después.

```
nombre_cliente = "Telcel"
total_solicitudes = 12348
```

A la izquierda del = va el nombre que tú inventas. A la derecha, el valor. Python no necesita que declares de qué "tipo" es — lo infiere solo.

0.2 — Tipos de datos básicos

- **Texto (`str`)**: va entre comillas. `"Telcel"`, `"2026-10-13"`.
- **Número entero (`int`)**: `12348`.
- **Número decimal (`float`)**: `0.5028344671201814`.
- **Verdadero/Falso (`bool`)**: `True`, `False`.
- **Nada (`None`)**: representa "no hay valor". Es distinto de `0` o de `""` (texto vacío).

Esto importa porque vas a ver mucho código que pregunta "¿esto es `None`?" antes de usar un dato — es la forma de Python de decir "¿este campo vino vacío de la API?".

0.3 — Listas

Una lista es una colección **ordenada** de valores, y pueden repetirse. Se escribe con corchetes `[]`:

```
columnas = ["Clave", "Prospecto", "Asesor"]
```

Puedes acceder a un elemento por su posición, empezando en 0:

```
primera_columna = columnas[0]  # "Clave"
```

0.4 — Diccionarios

Un diccionario asocia una **clave** a un **valor**, con llaves `{}` y dos puntos:

```
solicitud = {"Clave": "ABC123", "Estatus": "EVALUADO"}
```

Accedes al valor usando la clave entre corchetes, o con `.get()` (que no truena si la clave no existe, regresa `None` o un valor por defecto):

```
estatus = solicitud["Estatus"]           # "EVALUADO"
estatus = solicitud.get("Estatus", "")   # igual, pero seguro si no existe
```

Vas a ver diccionarios **todo el tiempo** en este proyecto, porque así es como llega la información cruda desde la API (JSON = diccionarios anidados).

0.5 — Sets (conjuntos)

Un set es como una lista, pero **sin orden y sin duplicados**. Se escribe también con llaves, pero sin los dos puntos:

```
columnas_requeridas = {"Clave", "Estatus", "Resultado"}
```

Su superpoder: la resta entre sets te dice qué le falta a uno respecto al otro. Lo vas a usar para validar que un archivo tenga todas las columnas que necesitas.

0.6 — Funciones

Una función es un bloque de código reutilizable. Se define con `def`, recibe parámetros entre paréntesis, y regresa un resultado con `return`:

```
def sumar(a, b):
    return a + b

resultado = sumar(3, 4)  # 7
```

Casi todo lo que vas a escribir en este proyecto son funciones pequeñas que hacen una cosa cada una: `descargar_solicitudes()`, `validar_esquema()`, `calcular_conversion()`, etc.

0.7 — Condicionales

`if` ejecuta código solo si una condición es verdadera:

```
if total_faltantes > 0.05:
    print("Hay demasiados faltantes")
else:
    print("Se puede continuar")
```

0.8 — Loops (`for`)

Un `for` repite una acción por cada elemento de una colección (lista, diccionario, etc.):

```
for dia in rango_de_dias:
    descargar_solicitudes(dia)
```

0.9 — Imports

Python no trae todo integrado — las herramientas que necesitas (pandas, requests, etc.) se "importan" al inicio del archivo:

```
import pandas as pd
import requests
```

El `as pd` es solo un apodo para no escribir pandas completo cada vez.

0.10 — Entorno virtual (venv) y pip

Antes de escribir una sola línea del proyecto, necesitas un espacio aislado donde instalar las librerías, para no mezclarlas con otros proyectos de tu computadora:

```
python -m venv venv
```

Esto crea una carpeta `venv/`. La activas (en Windows: `venv\Scripts\activate`; en Mac/Linux: `source venv/bin/activate`) y después instalas las librerías con `pip`:

```
pip install pandas openpyxl requests python-dotenv
```

Checklist antes de seguir a la Parte 1:

- ☐ Puedes explicar con tus palabras la diferencia entre una lista y un set
- ☐ Sabes escribir una función simple con `def` y `return`
- ☐ Tienes tu venv creado y activado
- ☐ Instalaste `pandas`, `openpyxl`, `requests`, `python-dotenv`

PARTE 1 — Las librerías del proyecto y para qué sirve cada una

Librería	Para qué la usas en ESTE proyecto
`requests`	Hacer las peticiones HTTP a la API de Nauphilus (login, descargar solicitudes, descargar comparativo)
`pandas`	Convertir listas de diccionarios en tablas, filtrar, agrupar, cruzar (merge), calcular las 6 tablas del ROI
`openpyxl`	Escribir el Excel final respetando fórmulas, colores y formato — pandas solo no puede hacer esto
`python-dotenv`	Leer usuario/contraseña desde un archivo `.env` en vez de dejarlos escritos en el código
`json`	Leer/escribir archivos de checkpoint (progreso guardado)
`datetime` / `calendar`	Manejar fechas, calcular el último día de un mes
`pathlib`	Checar si un archivo ya existe en disco (ej. "¿ya tengo la retro de este mes?")

No necesitas memorizar la documentación completa de cada una — en cada fase de abajo te digo exactamente qué función de cuál librería usar y cómo.

PARTE 2 — Arquitectura del proyecto

Organiza tu código en estos archivos desde el principio — no escribas todo en un solo script gigante:

```
telcel_roi/
|-- .env                      # usuario, contraseña, URL base (nunca se sube a git)
|-- clientes.json             # datos de conexión por cliente (ya lo tienes)
|-- config.py                 # sets y diccionarios de la Fase 0
|-- auth.py                   # login(), get_token()
|-- api_client.py             # get_solicitudes(), get_new_variable()
|-- parsers.py                # parse_solicitud(), extraer_campos()
|-- cli.py                    # preguntas al usuario: qué mes, checks de retro
|-- normalize.py              # limpieza, periodos, duplicados
|-- matching.py                # cruces (merge) y cálculo del % de faltantes
|-- metrics/
|   |-- conversion_costo.py
|   |-- resultados_aprobados.py
|   |-- tasas_beneficio.py
|   `-- utilidad_roi.py
|-- writer.py                  # escritura del Excel final con openpyxl
`-- main.py                    # junta todo, en orden
```

Cada archivo (`.py`) contiene funciones relacionadas entre sí. `main.py` es el único que "orquesta" — llama a las funciones de los demás archivos en el orden correcto.

PARTE 3 — FASE 0: Contrato de datos

Objetivo de esta fase: definir, antes de escribir cualquier otra cosa, qué columnas debe tener cada tabla del proyecto. Esto evita que a la mitad del proceso descubras que te falta un dato.

3.1 — De dónde salen los nombres de columna

No los inventas — los lees directo del lugar donde se generan los datos. En tu caso, las columnas de Solicitudes salen de lo que regresa la función que procesa cada solicitud cruda de la API (la función `return { ... }` que ya armaste). Lo mismo para el Comparativo.

Cómo lo haces tú: abre la función de descarga correspondiente, busca su línea `return {`, y copia cada nombre de clave tal cual aparece ahí.

3.2 — Escribe el set de columnas requeridas

En `config.py`:

```
COLUMNAS_SOLICITUDES_REQUERIDAS = {
    "Clave", "Prospecto", "Asesor", "Producto",
    "Ultima Modificación", "Estatus", "Resultado",
}

COLUMNAS_COMPARATIVO_REQUERIDAS = {
    "Propiedad", "Numero de Solicitud",
    "Monto Maximo Recomendado", "Folio Telcel",
}

COLUMNAS_RETRO_REQUERIDAS = {
    "estado_solicitud", "estatus_cobranza", "monto_acreditado",
}
```

Tres sets separados porque se usan en tres momentos distintos del pipeline (justo después de descargar Solicitudes, justo después de descargar Comparativo, y hasta que llega la retro de Telcel).

3.3 — Escribe la función de validación

En `config.py` también (o en un archivo nuevo `validaciones.py` si prefieres separarlo):

```
def validar_esquema(df, columnas_requeridas, nombre_tabla):
    faltantes = columnas_requeridas - set(df.columns)
    if faltantes:
        raise ValueError(f"Faltan columnas en {nombre_tabla}: {faltantes}")
```

El parámetro `nombre_tabla` es solo para que el mensaje de error te diga cuál tabla está incompleta — te ahorra tiempo de debugging después.

3.4 — Diccionario de nombres canónicos (solo para históricos)

Si vas a cargar los Excels históricos de meses pasados (que tienen nombres de columna distintos entre sí), necesitas un diccionario que traduzca ambos esquemas a un nombre único:

```
COLUMNAS_CANONICAS = {
    "Clave": "id_entrevista",
    "numero_solicitud_entrevista": "id_entrevista",
    "Folio": "request_id",
    "request_id": "request_id",
}
```

Se aplica así: `df = df.rename(columns=COLUMNAS_CANONICAS)`.

Checklist de Fase 0:

- ☐ 3 sets de columnas requeridas, uno por tabla, viviendo en `config.py`
- ☐ `validar_esquema()` escrita y probada (bórrale una columna a un DataFrame de prueba y confirma que truena con el mensaje correcto)
- ☐ Diccionario de nombres canónicos, solo si vas a usar archivos históricos

---

PARTE 4 — FASE 1: Descarga y preparación de la base

4.1 — Autenticación (`auth.py`)

Tu sistema (Nauphilus) usa un flujo llamado **OAuth con password grant**: mandas usuario y contraseña una vez, te regresan un "token" (una cadena de texto larga) que usas en cada petición siguiente, en vez de volver a mandar la contraseña.

Esto se hace con `requests.post()`:

```
import requests

def login(usuario, contrasena, base_url):
    respuesta = requests.post(
        f"{base_url}/auth/oauth/token",
        data={"grant_type": "password", "username": usuario, "password": contrasena},
    )
    respuesta.raise_for_status()  # truena si la petición falló (ej. contraseña mala)
    return respuesta.json()       # el token y cuánto tiempo dura, como diccionario
```

`raise_for_status()` es importante: sin ella, si la contraseña está mal, tu programa seguiría corriendo como si nada y fallaría más adelante de forma confusa.

4.2 — El token se vence — necesitas revisarlo

Guarda cuándo obtuviste el token (`time.time()` te da la hora actual en segundos) y cuánto dura (`expires_in`, que viene en la respuesta). Antes de cada petición, revisas si ya casi se vence (ej. le quedan menos de 5 minutos) y si es así, vuelves a hacer login.

4.3 — Pedir las solicitudes de un día

```
def get_solicitudes(fecha, base_url, membership_id, employee_id, token):
    url = (
        f"{base_url}/memberships/{membership_id}/core/creditApp"
        f"/getSolitudesByDates/{employee_id}/{fecha}/{fecha}"
    )
    respuesta = requests.get(url, headers={"Authorization": f"Bearer {token}"})
    respuesta.raise_for_status()
    return respuesta.json()  # una LISTA de diccionarios, uno por solicitud
```

El resultado es JSON crudo — información anidada y con muchos campos que no necesitas todavía.

4.4 — Aplanar cada solicitud a lo que sí necesitas

Esta función recibe UN diccionario crudo (una solicitud) y regresa un diccionario simple con solo 7-9 llaves (las que definiste en la Fase 0):

```
def parse_solicitud(item):
    return {
        "Clave": item.get("key", ""),
        "Estatus": item.get("status", {}).get("nombre", ""),
        "Resultado": item.get("result", ""),
        # ... el resto de campos, siguiendo el mismo patrón
    }
```

Fíjate en el patrón `item.get("status", {}).get("nombre", "")`: cuando un campo viene anidado (un diccionario dentro de otro), encadenas `.get()` — primero sacas el diccionario interno (`status`), luego el valor que quieres de ahí (`nombre`). El segundo argumento de cada `.get()` es el valor por defecto si no existe, para que nunca truene por un campo faltante.

4.5 — Convertir fechas (epoch → texto legible)

Las fechas de la API suelen venir como un número gigante (milisegundos desde 1970 — "epoch"). Se convierten así:

```
from datetime import datetime

fecha_legible = datetime.fromtimestamp(numero_epoch / 1000).strftime("%Y-%m-%d")
```

`/ 1000` porque la API da milisegundos y Python espera segundos. `.strftime(...)` da formato de texto a la fecha.

4.6 — De lista de diccionarios a tabla (DataFrame)

Una vez que tienes una lista de diccionarios (uno por solicitud, ya aplanados con `parse_solicitud`), la conviertes en tabla con una sola línea:

```
import pandas as pd
df = pd.DataFrame(lista_de_diccionarios)
```

Cada diccionario se vuelve una fila. Cada llave se vuelve una columna.

4.7 — Crear el periodo (mes canónico)

```
df["fecha"] = pd.to_datetime(df["Ultima Modificación"], errors="coerce")
df["periodo"] = df["fecha"].dt.to_period("M")
```

- `pd.to_datetime` convierte texto a un tipo de dato "fecha real" que pandas entiende.
- `errors="coerce"` significa: si una fecha viene mal escrita, no truenes — conviértela en "vacío" (NaT) para que la puedas detectar después.
- `.dt.to_period("M")` agrupa cualquier fecha de octubre 2026 bajo la misma etiqueta `2026-10`, sin importar el día exacto.

4.8 — Detectar y resolver duplicados

Antes de borrar nada, audita:

```
duplicados = df[df.duplicated(subset="Clave", keep=False)]
```

`subset="Clave"` dice "compara solo por esta columna". `keep=False` dice "muéstrame TODAS las copias, no solo la segunda en adelante" — así puedes ver el grupo completo de duplicados antes de decidir qué hacer.

Una vez que decides la regla (ej. "me quedo con la más reciente"):

```
df = df.sort_values("fecha").drop_duplicates(subset="Clave", keep="last")
```

`sort_values("fecha")` ordena por fecha primero, así `keep="last"` se queda con la más reciente de cada grupo.

Checklist de Fase 1:

- ☐ `login()` y manejo de vencimiento de token funcionando
- ☐ `get_solicitudes()` regresa una lista de diccionarios crudos para un día de prueba
- ☐ `parse_solicitud()` aplanado correctamente, confirmado contra 2-3 registros reales
- ☐ `pd.DataFrame()` construido sin errores
- ☐ `periodo` construido desde fecha real, no desde texto suelto
- ☐ Duplicados auditados ANTES de decidir la regla de limpieza

---

PARTE 5 — Capa interactiva: elegir el mes y checar la retro

Esta parte no existía en tus 2 scripts originales — la agregamos para que el programa se sienta como una herramienta terminada, no un script que hay que editar cada vez.

5.1 — Preguntar el mes

```
mes = int(input("¿Qué mes quieres procesar? (1-12): "))
anio = int(input("¿Qué año?: "))
```

`input()` siempre regresa texto — por eso lo envuelves en `int()` para convertirlo a número.

5.2 — Calcular el rango de fechas del mes automáticamente

```
import calendar
ultimo_dia = calendar.monthrange(anio, mes)[1]
fecha_inicio = f"{anio}-{mes:02d}-01"
fecha_fin = f"{anio}-{mes:02d}-{ultimo_dia}"
```

`calendar.monthrange(año, mes)` regresa dos datos: qué día de la semana empieza el mes, y cuántos días tiene. Por eso usamos `[1]` — el segundo valor. `{mes:02d}` da formato de 2 dígitos (`"03"` en vez de `"3"`).

5.3 — Checar si ya tienes la retro de ese mes

```
from pathlib import Path

ruta_retro = Path(f"retro/retro_{anio}-{mes:02d}.xlsx")
if ruta_retro.exists():
    print("Ya existe una retro guardada para este mes.")
else:
    print("Todavía no tienes la retro de Telcel para este mes.")
```

`Path(...).exists()` checa en el disco duro si el archivo ya está ahí, sin necesidad de abrirlo.

5.4 — Calcular el % de faltantes contra el umbral del 5%

Una vez que tienes (a) lo que le enviaste a Telcel y (b) lo que regresó, comparas:

```
comparacion = enviados[["Folio Telcel"]].merge(
    retro[["Folio Telcel"]], on="Folio Telcel", how="left", indicator=True
)
faltantes = (comparacion["_merge"] == "left_only").sum()
porcentaje_faltante = faltantes / len(comparacion)
```

```
if porcentaje_faltante > 0.05:
    mensaje = (
        f"Faltan {porcentaje_faltante:.1%} de respuestas — "
        "pide a Telcel que completen antes de seguir"
    )
    raise RuntimeError(mensaje)
```

`indicator=True` agrega una columna `_merge` que dice, por cada fila, si el folio se encontró en ambos lados (`"both"`) o solo en el primero (`"left_only"` — es decir, lo mandaste pero no tiene retro todavía).

Checklist de Fase 5 (capa interactiva):

- ☐ El programa pregunta mes/año en vez de que edites el código cada vez
- ☐ El rango de fechas se calcula solo, sin que captures el último día a mano
- ☐ `Path.exists()` te dice si ya tienes la retro antes de intentar procesarla
- ☐ El cálculo de % de faltantes detiene el programa si supera 5%, con un mensaje claro

---

PARTE 6 — FASE 2: Comparativo, envío a Telcel e integración de retro

6.1 — Pedir el Comparativo, uno por solicitud

A diferencia de Solicitudes (que viene de golpe para todo un día), el Comparativo se pide **individualmente** por cada solicitud ya evaluada:

```
def get_new_variable(credit_id, base_url, membership_id, token):
    url = (
        f"{base_url}/memberships/{membership_id}/core/creditApp"
        f"/{credit_id}/getNewVariable"
    )
    r = requests.get(url, headers={"Authorization": f"Bearer {token}"})
    if r.status_code == 200:
        return r.json()
    return None
```

Revisamos `status_code == 200` (200 significa "todo bien" en HTTP) en vez de usar `raise_for_status()` aquí, porque si UNA solicitud falla no quieres que se detenga todo el proceso — solo que esa quede como `None` y sigas con las demás.

6.2 — Procesar en lotes, con pausa

Si tienes 500 solicitudes evaluadas, no le pegas a la API 500 veces seguidas sin parar — la puedes saturar. Usas un loop con pausa:

```
import time

for i in range(0, len(ids), 80):  # de 80 en 80
    lote = ids[i:i+80]
    for credit_id in lote:
        datos = get_new_variable(credit_id, ...)
        # guardar datos
    time.sleep(2)  # pausa de 2 segundos entre lotes
```

`range(0, len(ids), 80)` genera números saltando de 80 en 80 — es la forma estándar de partir una lista larga en pedazos.

6.3 — Cruzar Solicitudes con Comparativo

```
base_integrada = solicitudes.merge(
    comparativo,
    left_on="Clave",
    right_on="Numero de Solicitud",
    how="left",
    validate="one_to_one",
)
```

`validate="one_to_one"` es la parte más importante de esta línea: le dice a pandas "truena si encuentras que una Clave tiene más de un Comparativo, o viceversa" — esto te protege de inflar tu base sin darte cuenta.

6.4 — Preparar el archivo que se manda a Telcel

```
columnas_para_telcel = [
    "Propiedad", "Numero de Solicitud", "Folio Telcel", "RFC", "Score",
]
columnas_que_llena_telcel = [
    "Estado solicitud", "Estatus cobranza", "Monto acreditado",
]

envio = base_integrada[columnas_para_telcel].copy()
for columna in columnas_que_llena_telcel:
    envio[columna] = pd.NA

envio.to_excel("envio_telcel.xlsx", index=False)
```

`pd.NA` crea una columna vacía (marcador de "falta dato") para que Telcel la llene. `.copy()` después de seleccionar columnas evita una advertencia común de pandas sobre modificar una "vista" en vez de una tabla independiente.

6.5 — Integrar la retro cuando llega

```
base_integrada["estado_solicitud"] = (
    retro["estado_solicitud"].combine_first(base_integrada["estado_solicitud"])
)
```

`combine_first` dice: "usa el valor de la izquierda si existe; si es nulo, usa el de la derecha". Aquí priorizas lo que llega de Telcel sobre lo que ya tenías. Repites esta línea para cada campo que puede actualizarse — nunca en bloque.

Checklist de Fase 2:

- ☐ `get_new_variable` maneja el caso de que una solicitud falle sin tronar todo el programa
- ☐ Procesamiento por lotes con pausa implementado
- ☐ `merge` con `validate="one_to_one"` confirmado sin errores
- ☐ Archivo de envío a Telcel generado con columnas vacías correctas
- ☐ `combine_first` aplicado campo por campo tras recibir la retro

PARTE 7 — FASE 3: Tabla 1 (Conversión) y Tabla 2 (Costo)

7.1 — Contar únicos, no filas

```
iniciados = base_integrada.groupby("periodo")["Clave"].nunique()
```

`groupby("periodo")` agrupa las filas por mes. `["Clave"].nunique()` cuenta cuántas Claves **distintas** hay en cada grupo. Usamos `nunique()` y no `len()` o `.size()` porque si una Clave se repitiera, esas funciones la contarían de más.

7.2 — Filtrar antes de agrupar, para "evaluados"

```
evaluados = (
    base_integrada[base_integrada["Estatus"] == "EVALUADO"]
    .groupby("periodo")["Clave"]
    .nunique()
)
```

`base_integrada["Estatus"] == "EVALUADO"` crea una lista de True/False, una por fila. Ponerla dentro de `base_integrada[...]` te deja solo las filas donde es True.

7.3 — Calcular la conversión sin dividir entre cero

```
# ojo: la fórmula correcta es evaluados / iniciados, no al revés
conversion = evaluados.div(iniciados).where(iniciados != 0)
```

`.div()` es como escribir / pero como método, útil quando divides series completas. `.where(condición)` dice: "deja el resultado solo donde la condición es verdadera; donde no, pon vacío (NaN)" — así nunca te sale un error de división entre cero.

7.4 — Costos desde la hoja de facturación

```
costos = pd.read_excel("plantilla.xlsx", sheet_name="Data Facturas Sara")
```

`sheet_name` le dice a pandas de cuál pestaña del Excel leer — un Excel puede tener varias hojas, y sin este parámetro pandas solo lee la primera.

Checklist de Fase 3:

- ☐ `nunique()` usado, confirmado que no es lo mismo que `.size()` en tu caso
- ☐ Filtro de evaluados aplicado ANTES del `groupby`
- ☐ División protegida con `.where()`
- ☐ `sheet_name` correcto al leer la hoja de costos

PARTE 8 — FASE 4: Tabla 3 (Resultados) y Tabla 4 (Aprobados)

8.1 — Normalizar texto antes de comparar

```
base_integrada["resultado_normalizado"] = (
    base_integrada["Resultado"].str.strip().str.lower()
)
```

`.str.strip()` quita espacios al inicio/final. `.str.lower()` vuelve todo minúsculas. Haces esto porque el mismo valor puede venir escrito distinto ("Baja Riesgo 5" vs "baja riesgo 5") y sin normalizar, pandas los trataría como cosas diferentes.

8.2 — Mapear a categorías fijas

```
mapa = {
    "mantiene riesgo": "riesgo_6",
    "baja riesgo 5": "riesgo_5",
    "baja riesgo 4": "riesgo_4",
```

```
}
base_integrada["grupo_riesgo"] = base_integrada["resultado_normalizado"].map(mapa)
```

`.map(diccionario)` busca cada valor de la columna dentro del diccionario y lo reemplaza por su valor correspondiente. Si un valor no está en el diccionario, el resultado es `None` — por eso siempre revisas después cuáles valores no se reconocieron:

```
sin_reconocer = base_integrada["grupo_riesgo"].isna()
no_reconocidos = base_integrada[sin_reconocer]["Resultado"].value_counts()
```

`.value_counts()` te cuenta cuántas veces aparece cada valor distinto — útil para ver de un vistazo qué texto nuevo se te coló.

8.3 — Convertir filas largas en columnas (pivotear)

```
tabla_resultados = (
    base_integrada.dropna(subset=["grupo_riesgo"])
    .groupby(["periodo", "grupo_riesgo"])["Clave"].nunique()
    .unstack(fill_value=0)
    .reindex(columns=["riesgo_4", "riesgo_5", "riesgo_6"], fill_value=0)
)
```

- `.dropna(subset=["grupo_riesgo"])` descarta las filas sin categoría reconocida.
- `.groupby(["periodo", "grupo_riesgo"])` agrupa por DOS columnas a la vez.
- `.unstack(fill_value=0)` convierte la segunda columna del agrupamiento (`grupo_riesgo`) en columnas de la tabla final, en vez de filas. `fill_value=0` rellena con cero las combinaciones que no existan.
- `.reindex(columns=[...], fill_value=0)` fuerza que las 3 columnas existan siempre, aunque algún mes no tenga casos de una categoría.

8.4 — Filtrar aprobados

```
aprobados = base_integrada[base_integrada["estado_solicitud"].str.upper() == "APRO"]
```

Repites el mismo patrón de pivoteo de arriba, pero sobre este subconjunto filtrado.

Checklist de Fase 4:

- ☐ Normalización de texto aplicada antes de mapear
- ☐ Categorías no reconocidas revisadas con `value_counts()`, no ignoradas
- ☐ `reindex` fuerza las 3 columnas de riesgo siempre presentes
- ☐ Filtro de aprobados usa `estado_solicitud`, no `estatus_cobranza`

PARTE 9 — FASE 5: Tabla 5 (Tasas) y Tabla 6 (Beneficio)

9.1 — Alinear dos tablas con índices distintos

```
aprobados_align, resultados_align = tabla_aprobados.align(
    tabla_resultados, join="outer", axis=0, fill_value=0
)
```

`.align()` hace que dos tablas terminen con exactamente los mismos periodos en sus filas, rellenando con cero donde a una le falte un mes que la otra sí tiene.

9.2 — Tasa combinada: suma primero, divide después

```
aprobados_45 = aprobados_align["riesgo_4"] + aprobados_align["riesgo_5"]
resultados_45 = resultados_align["riesgo_4"] + resultados_align["riesgo_5"]
tasa_45 = aprobados_45.div(resultados_45).where(resultados_45 != 0)
```

Este es el punto donde más fácil se equivocan quienes automatizan esto: **no promedies las tasas individuales de riesgo 4 y 5** — suma los numeradores entre sí, suma los denominadores entre sí, y DESPUÉS divide. Es aritmética básica, no pandas, pero es donde más se rompe la lógica si la saltas.

9.3 — Monto promedio y detección de datos faltantes

```
metricas = aprobados.groupby("periodo").agg(
    aprobados_ids=("Clave", "nunique"),
    montos_validos=("monto_acreditado", "count"),
    monto_promedio=("monto_acreditado", "mean"),
)
```

`.agg(nombre_salida=("columna", "operación"))` te deja calcular varias cosas distintas en una sola llamada, cada una con su propio nombre de columna resultante. Comparas `aprobados_ids` contra `montos_validos`: si son diferentes, significa que hay aprobados sin monto registrado.

Checklist de Fase 5:

- ☐ `.align()` usado antes de combinar las dos tablas
- ☐ Tasa combinada calculada como razón de sumas, NO como promedio de tasas
- ☐ `.agg()` con nombres de salida personalizados
- ☐ Diferencia entre `aprobados_ids` y `montos_validos` revisada

PARTE 10 — FASE 6: Utilidad y ROI

10.1 — Encadenar cálculos con `.assign()`

```
resumen = resumen.assign(
    fraude=lambda d: d["beneficio"] * 0.12,
    mora=lambda d: d["beneficio"] * 0.20,
    neto=lambda d: d["beneficio"] * 0.80,
)
resumen["utilidad"] = resumen["neto"] - resumen["fraude"] - resumen["mora"]
```

`.assign()` agrega columnas nuevas a una tabla sin modificar la original directamente (regresa una copia con las columnas agregadas). Cada `lambda d: ...` es una mini-función de una sola línea que recibe la tabla tal como va hasta ese punto — así `mora` y `neto` se calculan ambos a partir de `beneficio`, sin que uno dependa del otro.

10.2 — Verificación automática

```
import numpy as np
assert np.isclose(resumen["utilidad"], resumen["beneficio"] * 0.48, rtol=1e-9).all()
```

`np.isclose()` compara dos números con una tolerancia (porque los decimales de computadora casi nunca son exactamente iguales). `.all()` dice "esto debe ser cierto para TODAS las filas". `assert` detiene el programa inmediatamente si la condición es falsa — es tu propio sistema de alarma para detectar un error en la fórmula antes de que llegue al Excel final.

10.3 — ROI, protegido contra costo cero

```
roi_bruto = (resumen["utilidad"] - resumen["costo_nau"]) / resumen["costo_nau"]
resumen["roi"] = roi_bruto.where(resumen["costo_nau"] != 0)
```

Checklist de Fase 6:

- ☐ `.assign()` con las 3 fórmulas (fraude, mora, neto)
- ☐ `assert` de verificación (48%) corrido y pasando
- ☐ ROI protegido contra división entre cero
- ☐ ROI se guarda como decimal (`5.67`), nunca multiplicado por 100

PARTE 11 — FASE 7: Generar el Excel final

11.1 — Trabajar siempre sobre una copia

```
from shutil import copy2
copy2("plantilla.xlsx", "salida_octubre.xlsx")
```

Nunca abras ni modifiques el archivo original — si algo sale mal, lo pierdes para siempre.

11.2 — Abrir preservando fórmulas

```
from openpyxl import load_workbook
libro = load_workbook("salida_octubre.xlsx", data_only=False, keep_links=True)
hoja = libro["Tablas"]
```

`data_only=False` es la parte clave: así ves y conservas las fórmulas de Excel en vez de solo sus últimos valores calculados.

11.3 — Escribir solo las celdas que cambian

```
hoja["C5"] = iniciados_octubre
hoja["C6"] = evaluados_octubre
```

Nunca uses `df.to_excel()` sobre esta hoja — te borraría colores, bordes, celdas combinadas y fórmulas existentes. Escribes celda por celda, a mano, como si estuvieras tecleando en Excel.

11.4 — Formato de presentación

```
hoja["Q9"].number_format = "0.00%"
```

Esto solo cambia cómo SE VE el número en Excel — el valor guardado sigue siendo decimal (`5.67`), Excel lo muestra como 567%.

11.5 — Guardar y verificar

```
libro.save("salida_octubre.xlsx")

# Verificación 1: las fórmulas siguen ahí
libro_formulas = load_workbook("salida_octubre.xlsx", data_only=False)

# Verificación 2: los valores (ADVERTENCIA: openpyxl no recalcula fórmulas nuevas)
libro_valores = load_workbook("salida_octubre.xlsx", data_only=True)
```

Si agregaste fórmulas NUEVAS (no solo llenaste celdas que ya tenían fórmula), tienes que abrir el archivo una vez en Excel/LibreOffice y guardarlo ahí, para que se recalculen — openpyxl por sí solo nunca ejecuta una fórmula, solo la escribe o la lee.

Checklist de Fase 7:

- ☐ Se trabaja siempre sobre copia
- ☐ `data_only=False` + `keep_links=True` al abrir
- ☐ Escritura celda por celda, nunca `to_excel` sobre la hoja de presentación
- ☐ Formato de porcentaje aplicado donde corresponde
- ☐ Doble verificación tras guardar (fórmulas y valores)
- ☐ Archivo probado abriéndolo manualmente en Excel

PARTE 12 — Glosario rápido de funciones (para consulta)

Función	Librería	Qué hace
`requests.get()` / `.post()`	requests	Hace una petición HTTP (traer o enviar datos a un servidor)
`.raise_for_status()`	requests	Truena si la petición falló
`pd.DataFrame()`	pandas	Convierte lista de diccionarios en tabla
`pd.to_datetime()`	pandas	Convierte texto en fecha real
`.dt.to_period("M")`	pandas	Agrupa fechas por mes
`.duplicated()`	pandas	Marca filas repetidas
`.drop_duplicates()`	pandas	Elimina filas repetidas según una regla
`.groupby()`	pandas	Agrupa filas por el valor de una o más columnas
`.nunique()`	pandas	Cuenta valores distintos dentro de un grupo
`.merge()`	pandas	Cruza dos tablas (como un VLOOKUP, pero más potente)
`.combine_first()`	pandas	Rellena valores vacíos con los de otra columna/tabla
`.unstack()`	pandas	Convierte categorías de fila en columnas
`.reindex()`	pandas	Fuerza que ciertas columnas/filas existan siempre
`.align()`	pandas	Hace que dos tablas compartan el mismo índice
`.assign()`	pandas	Agrega columnas calculadas a una tabla
`.where()`	pandas	Reemplaza con vacío donde una condición es falsa
`load_workbook()`	openpyxl	Abre un Excel conservando formato y fórmulas
`.number_format`	openpyxl	Cambia cómo se ve un número (sin cambiar su valor)
`Path.exists()`	pathlib	Revisa si un archivo ya existe en disco
`calendar.monthrange()`	calendar	Da el último día de un mes

Cierre

Si completaste los 8 checklists de este documento, en ese orden, tienes el programa completo funcionando: desde pedirle el mes al usuario, pasando por la descarga autenticada, el cruce con Telcel, las 6 tablas, hasta el Excel final con formato intacto.

No hay atajos honestos para saltarse fases — cada una depende de que la anterior esté validada con sus números reales. Si en algún punto un checklist no te cuadra, no sigas a la siguiente fase: ahí está el bug, y es más barato encontrarlo ahí que en la Fase 7.